Multi-Robot Double-Angle Formation Controller

Mathematical Formulation and Data Logging Reference

TurtleBot3 Swarm Experiments

August 15, 2026

1 Overview

This document describes the controller implemented in `multi_unicycle_controller_double_angle.py`, which drives a group of $N = 4$ TurtleBot3 unicycle robots into and around a desired formation while simultaneously enforcing two geometric constraints on every controlled inter-robot pair:

- **Connectivity**: neighbors must not drift further apart than a connectivity radius R_c (otherwise they lose line-of-sight / sensing range).
- **Safety**: neighbors must not come closer together than a safety radius R_s (otherwise their physical bodies risk collision).

These constraints are enforced using a *double-angle visual-cone* parameterization of inter-agent bearing angles, which converts the distance constraints $R_s + r \leq \|p_i - p_j\| \leq R_c - r$ into equivalent bounds on an angle ψ_{ij} that remains well-conditioned (no singularities) across the full admissible range.

This document also specifies exactly what data the controller now logs to CSV (replacing the previous GIF/animation export, which was slow and memory-heavy for long runs) and what each column means.

2 Unicycle Model

Each robot i is modeled as a unicycle with state (x_i, y_i, θ_i) in the global frame and control inputs (v_i, ω_i):

$$\dot{x}_i = v_i \cos\theta_i \tag{1}$$

$$\dot{y}_i = v_i \sin\theta_i \tag{2}$$

$$\dot{\theta}_i = \omega_i \tag{3}$$

Odometry feedback provides (x_i, y_i, θ_i) directly, and body-frame velocities (v_i, ω_i) are rotated into global-frame velocities $(v_{x,i}, v_{y,i}, \omega_i)$ for logging purposes.

3 Formation Graph and Desired Geometry

The 4 robots are connected by 5 edges, encoded in the incidence matrix $H \in \mathbb{R}^{5\times 4}$:

$$H = \begin{pmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \qquad \bar{H} = H \otimes I_2 \in \mathbb{R}^{10\times 8} \tag{4}$$

Row k of H defines edge k between the two robots whose entries are -1 and $+1$. With $p = [x_1\, y_1\, x_2\, y_2\, x_3\, y_3\, x_4\, y_4]^\top$ the stacked global positions, the edge (relative-position) vector for all edges is

$$e = \bar{H}\, p \in \mathbb{R}^{10}, \qquad e_k = p_j - p_i \text{ for edge } k = (i, j). \tag{5}$$

A desired formation shape $p^\star$ (see `self.pstar` in code) defines the desired edge vectors $e^\star = \bar{H}p^\star$, and the desired Euclidean distance for edge k is

$$d_k^\star = \|e_k^\star\|. \tag{6}$$

A second incidence matrix H_M (with $\bar{H}_M = H_M \otimes I_2$) defines an additional pair used purely for collision-avoidance monitoring between non-adjacent robots.

4 Double-Angle Visual-Cone Parameterization

4.1 Physical radii and their relationship to the bearing angle

Let r be the physical radius of each robot footprint, R_s the safety margin, R_c the connectivity (sensing) radius, and R_{crit} the critical (hard) collision radius. For a pair of robots whose *centers* are a distance $d_{ij} = \|e_{ij}\|$ apart, define the half-angle subtended by one robot's footprint as seen from the other:

$$\psi_{ij} = 2\arcsin\left(\frac{r}{d_{ij}}\right). \tag{7}$$

This angle increases monotonically as d_{ij} decreases, so it is used as a proxy for distance that behaves well (no division blow-up) as robots get close. Three reference angles bound the admissible range of ψ_{ij}, each obtained from the corresponding critical distance via the double-angle identity $\cos\psi = 1 - 2\sin^2(\psi/2) = 1 - 2(r/d)^2$:

$$\bar{R}_c = 1 - 2\left(\frac{r}{R_c - r}\right)^2, \qquad \psi_{\min} = \arccos(\bar{R}_c) \tag{8}$$

$$\bar{R}_s = 1 - 2\left(\frac{r}{R_s + r}\right)^2, \qquad \psi_{\max} = \arccos(\bar{R}_s) \tag{9}$$

$$\bar{R}_{crit} = 1 - 2\left(\frac{r}{R_{crit}}\right)^2, \qquad \psi_{crit} = \arccos(\bar{R}_{crit}) \tag{10}$$

4.2 Distance bounds used for logging

Every logged inter-robot distance d_{ij} is compared against the same pair of physical bounds used to derive the angle limits above:

$$\boxed{\text{lower bound } = R_s + r} \qquad \boxed{\text{upper bound } = R_c - r} \tag{11}$$

Why these bounds: R_s and R_c are specified as clearances/ranges measured from each robot's *surface*, not its center. Since each robot occupies a disk of radius r, the minimum center-to-center distance that still preserves a safety clearance of R_s between the two disks' surfaces is R_s+r, and the maximum center-to-center distance that still keeps the two robots' surfaces within a connectivity range of R_c is $R_c - r$. In the current configuration ($r = 0.2\,\text{m}$, $R_s = 0.4\,\text{m}$, $R_c = 3.0\,\text{m}$), this gives:

$$\text{lower bound} = 0.4 + 0.2 = 0.6\,\text{m}, \qquad \text{upper bound} = 3.0 - 0.2 = 2.8\,\text{m}. \tag{12}$$

Every edge's desired distance $d_k^\star$ (derived from the desired formation $p^\star$) is chosen to lie strictly within this interval, and the controller is designed so that, starting from a feasible configuration, the actual distance $d_k(t)$ remains within (lower bound, upper bound) for all t.

4.3 Control law

For each formation edge k with bearing-derived quantities ψ_k, unit vector $g_k = e_k/d_k$, and left/right rotations of g_k by $\pm\psi_k/2$:

$$R(\pm\psi_k/2) = \begin{pmatrix} \cos(\psi_k/2) & \mp\sin(\psi_k/2) \\ \pm\sin(\psi_k/2) & \cos(\psi_k/2) \end{pmatrix}, \qquad g_k^R = R(-\psi_k/2)\, g_k, \quad g_k^L = R(+\psi_k/2)\, g_k \tag{13}$$

a projected gradient direction is built from the projector $P_g = I - gg^\top$:

$$f_{g,k} = P_{g_k^L}\, g_k^R + P_{g_k^R}\, g_k^L, \qquad f_{\psi,k} = \frac{\sin(2\psi_k)}{2(1+\cos\psi_k)}. \tag{14}$$

A barrier-like gain ξ_k pushes the edge back toward $d_k^\star$ as ψ_k approaches either limit angle, using the error term $E_k = \cos^2\psi_k - \cos^2\psi_k^\star$ and the corresponding one-sided gain (K_c near the connectivity limit, K_s near the safety limit):

$$\xi_k = \frac{4E_k}{K_k^2 - E_k^2}, \qquad K_k = q\,K_{c,k} + (1-q)\,K_{s,k}, \qquad q = \mathbb{1}[E_k > 0]. \tag{15}$$

The aggregate formation velocity in the stacked position space is

$$\dot{p} = -\bar{H}^\top R_\psi\, \xi, \tag{16}$$

where $R_\psi = \text{blockdiag}(f_{\psi,1} f_{g,1}, \dots)$. This is then converted, per robot i, into unicycle commands using the current heading θ_i and a small look-ahead length ℓ:

$$v_i = 2\,(\dot{p}_{2i}\cos\theta_i + \dot{p}_{2i+1}\sin\theta_i) \tag{17}$$

$$\omega_i = \frac{-\dot{p}_{2i}\sin\theta_i + \dot{p}_{2i+1}\cos\theta_i}{\ell} \tag{18}$$

clipped to $v_i \in [-v_{max}, v_{max}]$ and $\omega_i \in [-\omega_{max}, \omega_{max}]$. An analogous but separate term ξ_β (using R_{crit}) is computed for pairwise collision avoidance and can optionally be added to $\dot{p}$ (disabled by default in the current code).

5 Data Logging

5.1 What changed and why

The controller previously rendered a combined trajectory + distance `matplotlib` animation and saved it as a GIF (`animate_combined`). For long experiments this was slow to render and consumed large amounts of RAM (every frame is a rasterized image held in memory before encoding). This has been replaced with a lightweight CSV export (`save_csv`) that streams the already-collected numeric log into a single file with negligible memory and CPU cost. The PNG summary plots (final trajectories, distance-vs-time) are still produced, since those are cheap.

5.2 Sampling

State is logged every `log_interval` = 0.1 s of wall-clock time (see `log_trajectories`), once all four robots have produced at least one odometry message.

5.3 CSV schema

The exported file (`log_data.csv`, written into the timestamped output folder alongside the PNG plots) has one row per logged tick and the following columns:

Column(s)	Description
`time`	Elapsed time since logging started (s)
`bot_i_x`, `bot_i_y`, `bot_i_theta`	Global-frame pose of robot i for $i \in \{0, 1, 2, 3\}$ (m, m, rad)
`d1` ... `d5`	Actual inter-robot distance for formation edges 1–5 (m)
`e1` ... `e5`	Distance error $e_k = d_k - d_k^\star$ for edges 1–5 (m)
`lower_bound`	Constant column, $R_s + r$ (m) — minimum admissible distance
`upper_bound`	Constant column, $R_c - r$ (m) — maximum admissible distance

Edges 1–5 correspond to rows 1–5 of H, i.e. the robot pairs (bot_0–bot_1), (bot_1–bot_2), (bot_2–bot_3), (bot_3–bot_0), (bot_1–bot_3) respectively (see `get_connected_pairs`).

`lower_bound` and `upper_bound` are repeated on every row (rather than stored once) so that the CSV can be loaded directly into a plotting tool (e.g. `pandas` + `matplotlib`) and the bounds drawn as reference lines against `d1`...`d5` or `e1`...`e5` without any extra bookkeeping.

5.4 Output directory layout

On shutdown (`save_all_plots`), the controller writes to `~/turtlebot3_simulations/plots/<timestamp>/`:

```
~/turtlebot3_simulations/plots/<YYYY-MM-DD_HH-MM-SS>/
    trajectory.png # final trajectories + initial/final connections
    distances.png # d_k(t) vs. desired distance, per edge
    log_data.csv # full numeric log (see schema above)
```

5.5 Post-processing example

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("log_data.csv")

fig, ax = plt.subplots()
for k in range(1, 6):
    ax.plot(df["time"], df[f"d{k}"], label=f"d{k}")
ax.axhline(df["lower_bound"].iloc[0], color="red", linestyle="--", label="lower␣bound␣(
    Rs+r)")
ax.axhline(df["upper_bound"].iloc[0], color="gray", linestyle="--", label="upper␣bound␣(
    Rc-r)")
ax.set_xlabel("time␣(s)"); ax.set_ylabel("distance␣(m)"); ax.legend()
plt.show()
```